

DSA02- COMPUTER VISION WITH OPENCV
ASSIGNMENT

A. Assignment Information

Department:	(AI & ML)
Programme:	FACULTY OF ENGINEERING
Course Code & Course Name	DSA0204- COMPUTER VISION WITH OPENCV
Academic Year / Batch	2026-2027
Faculty Name	Dr. G. A. SENTHIL
Assignment Title	Analyse and design a computer vision–based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.
Date of Issue	20-08-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) - CO	CO1: Implement fundamental image processing techniques required for computer vision and understand the image formation process. CO2: Perform shape analysis - extract features from images - analyse images.

	CO3: Generate 3D models from images. CO4: Develop applications using computer vision techniques. CO5: Understand video processing - motion computation - 3D vision - geometry.
Bloom's Taxonomy Level	. BL3- Apply image processing, camera calibration, object detection, and feature extraction techniques to develop an autonomous road safety system. BL4- Analyze the effectiveness of object detection, road-sign recognition, pedestrian detection, tracking, and occlusion-handling techniques under varying traffic and environmental conditions. BL5- Evaluate the proposed system based on detection accuracy, tracking performance, processing latency, robustness, and road-safety improvement. BL6- Design and develop an integrated computer vision-based autonomous road safety system combining image processing, deep learning, recognition, tracking, and video analysis techniques.
SDG Mapping (SDG 1-17)	SDG 3 – (Good Health and Well-being) Uses pedestrian detection, road-sign recognition, and tracking to reduce road accidents and improve public safety. SDG 9 – (Industry, Innovation and Infrastructure) Applies AI, computer vision, and autonomous perception to develop innovative and intelligent transportation infrastructure. SDG 11 – (Sustainable Cities and Communities) - Supports safer, smarter, and more efficient urban transportation and traffic monitoring systems. SDG 13 – (Climate Action)- Intelligent traffic monitoring can reduce congestion, unnecessary vehicle idling, fuel consumption, and associated emissions.
Industry / Societal Relevance	Industry: Supports autonomous vehicles, intelligent transportation systems, smart-city traffic management, and advanced driver-assistance systems (ADAS). Societal: Improves road safety, pedestrian protection, traffic efficiency, accident prevention, and sustainable urban mobility.

Assignment Question

Analyse and design a computer vision–based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.

Assignment Information

The uploaded assignment brief identifies the course as DSA02 – Computer Vision with OpenCV, under the Department of Artificial Intelligence and Machine Learning, Faculty of Engineering, for the 2026–2027 academic year. The stated assignment requires analysis and design of an autonomous road-safety system combining camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. It also requires evaluation using detection accuracy, tracking performance, processing latency, and road-safety improvement, followed by discussion of intelligent transportation and public-safety relevance

The brief maps the work to CO1–CO5 and Bloom’s levels BL3–BL6, covering image processing, shape analysis, 3D vision, computer-vision application development, video processing, motion computation, geometry, analysis, evaluation, and system design.

The brief identifies industry relevance in autonomous vehicles, intelligent transportation systems, smart-city traffic management and ADAS, and societal relevance in pedestrian protection, accident prevention, traffic efficiency and sustainable urban mobility.

Report Contents

1. Problem Statement and Problem Formulation
2. Objective and Expected Outcomes
3. Requirements, Constraints and Assumptions
4. Application of Relevant Course Knowledge / Concepts
5. Design / Proposed Solution / Methodology
6. Algorithm / Pseudocode / Flowchart
7. Implementation / Source Code and Environment / Tools Used
8. Test Cases and Expected / Actual Results
9. Execution Screenshots / Outputs

10. Results and Validation
11. Analysis, Comparison, Trade-offs and Justification
12. Broader Considerations / SDG Relevance
13. Conclusion, Limitations and Possible Improvements
14. Individual Contribution of Group Members
15. References
16. Individual Contribution of Group Members – Declaration

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

1.1 Background of the Problem

Road transportation contains a large number of moving and stationary elements that must be understood quickly. A normal road scene can contain cars, buses, trucks, motorcycles, bicycles, pedestrians, traffic signs, lane boundaries, zebra crossings, road arrows, barriers and unexpected obstacles. Their positions change continuously, and several objects can overlap from the camera viewpoint. A road-safety system therefore needs both spatial understanding and temporal understanding.

Conventional safety mechanisms may use human observation, traffic signals, fixed cameras or individual vehicle sensors. These mechanisms are useful, but a vision-based intelligent system can add automated scene interpretation. Modern object-detection and tracking algorithms can transform video frames into machine-readable information. This makes it possible to study road-user behavior, detect vulnerable road users, recognize traffic information and monitor potentially hazardous situations.

1.2 Detailed Problem Statement

The central problem addressed by this project is the design of a computer vision pipeline that can observe a road scene through a camera and automatically identify important objects and road information with acceptable accuracy and processing speed. The proposed system should not depend on a single detection function. Instead, it should integrate camera calibration, object detection, pedestrian detection, traffic-sign recognition, road-marking recognition, multi-object tracking and occlusion handling.

Camera calibration is included because lens distortion can affect the geometric appearance of the scene. Object and pedestrian detection is required to identify road users. Sign and marking recognition provides semantic information about traffic rules and road structure. Tracking is required because a sequence of independent detections does not indicate whether an object in frame t is the same object observed in frame $t+1$. Occlusion handling is required because road users can temporarily disappear behind other objects.

The system must also be analyzed under different traffic and environmental conditions. A model that performs well only in clear daytime traffic may not be sufficiently robust for a practical road-safety application. Therefore, the project includes controlled comparison of traffic density, illumination, visibility and occlusion conditions.

1.3 Problem Formulation

Let the input road video be represented as $F = \{F_1, F_2, \dots, F_T\}$, where F_t represents the image captured at time t . For every frame, the system produces detections $D(t)$. Each detection can be represented as $D = [\text{class}, x, y, \text{width}, \text{height}, \text{confidence}]$. The tracking module associates detections across frames and generates tracked objects $O(t) = [\text{class}, \text{bounding box}, \text{confidence}, \text{track ID}]$.

The design objective is to maximize the reliability of road-scene perception while minimizing processing latency. A useful conceptual objective is to maximize detection quality and track continuity subject to the available computational resources and a practical latency limit.

1.4 Core Problems to be Solved

- Correct distortion introduced by the camera lens.
- Detect vehicles, pedestrians and other selected road users.
- Recognize selected traffic signs and road markings.
- Maintain unique identities for multiple moving objects.
- Reduce identity loss during partial or temporary occlusion.
- Operate under different traffic densities.
- Handle changes in brightness, shadows, haze, rain or low-light conditions where the dataset permits.
- Maintain an acceptable balance between detection accuracy and processing speed.
- Generate quantitative evidence through detection, tracking and latency metrics.
- Study the practical value and limitations of the system for intelligent transportation and public safety.

1.5 Input–Process–Output Formulation

Stage	Description
Input	Forward-facing road video or image sequence.
Calibration	Estimate/use camera parameters and correct lens distortion.
Preprocessing	Resize, color conversion and optional lightweight enhancement.
Detection	YOLO detects road objects and pedestrians.
Recognition	Traffic signs and road markings are identified.
Tracking	ByteTrack associates detections across frames.

Occlusion Handling	Temporarily missing tracks are managed using temporal continuity.
Safety Analysis	Rule-based safety indicators are calculated.
Output	Annotated video, object IDs, recognized road information and metrics.

1.6 Problem Scope

The scope is a research and academic prototype rather than a certified autonomous driving controller. The system focuses on visual perception and performance evaluation. It does not attempt to control steering, braking or acceleration. Any real deployment would require additional sensors, validated safety logic, redundancy and regulatory certification.

1.7 Research Questions

1. How accurately can the system detect vehicles and pedestrians?
2. How effectively can it recognize selected road signs and road markings?
3. How reliably can ByteTrack preserve object identities in crowded scenes?
4. How does partial occlusion affect detection and tracking?
5. How do illumination and weather-like conditions influence performance?
6. What processing latency can be achieved on the selected hardware?
7. Which configuration provides the best accuracy–speed trade-off?
8. Can timely identification of safety-relevant events be improved compared with a simple detection-only baseline?

2. OBJECTIVE AND EXPECTED OUTCOMES

2.1 Main Objective

To design, implement and evaluate a near-real-time computer vision–based autonomous road safety system that integrates camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking and occlusion handling.

2.2 Specific Objectives

- Develop a modular road-scene video-processing pipeline.
- Apply camera calibration concepts to reduce geometric distortion.
- Detect important road users using YOLO.
- Detect pedestrians as a priority vulnerable-road-user class.
- Recognize selected traffic signs.
- Recognize selected road markings and lane boundaries.
- Track multiple objects using ByteTrack.
- Maintain tracks during short-term partial occlusion.
- Measure precision, recall and F1-score.
- Measure tracking continuity, ID switches and, when possible, standard MOT metrics.
- Measure average frame-processing latency and FPS.
- Compare performance across traffic and environmental conditions.
- Assess road-safety improvement using defined experimental proxy measures.

2.3 Expected Outcomes

The expected outcome is a working prototype capable of accepting road video and generating annotated frames. Each relevant road object should receive a class label and, where tracking is active, a stable track ID. Recognized signs and markings should be displayed clearly. The system should produce a metric table and graphs showing how performance changes across test conditions.

2.4 Academic Outcomes

- Understanding of camera geometry and calibration.
- Practical use of OpenCV.
- Experience with deep-learning inference using PyTorch.
- Understanding of YOLO object detection.
- Understanding of multi-object tracking using ByteTrack.
- Experience in dataset preparation and evaluation.

- Ability to compare models using quantitative evidence.
- Ability to analyze engineering trade-offs.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional Requirements

- The system shall load road-scene video or receive frames from a camera.
- The system shall correct or account for camera distortion.
- The system shall detect selected road users.
- The system shall recognize selected traffic signs.
- The system shall recognize selected road markings/lane regions.
- The system shall track multiple objects.
- The system shall manage temporary track loss.
- The system shall display or save annotated outputs.
- The system shall record performance measurements.

3.2 Hardware Requirements

- Computer or cloud runtime.
- Camera or suitable road-scene video.
- GPU recommended for efficient deep-learning inference.
- Suitable RAM and storage for video and model files.
- Optional CUDA-compatible GPU for accelerated local execution.

3.3 Software Requirements

Tool	Purpose
Python	Main programming language
OpenCV	Video processing, calibration and visualization
YOLO	Object and pedestrian detection
PyTorch	Deep-learning model execution
ByteTrack	Multi-object tracking
NumPy	Numerical array processing
Pandas	Metric organization

Matplotlib	Graphs and result visualization
Google Colab	Cloud execution and GPU experimentation

3.4 Dataset Requirements

The selected dataset should contain road-scene images or video with suitable object labels. If sign and road-marking recognition are included, the dataset must contain those classes or separate datasets should be used. The report should document the exact dataset name, source, number of samples, classes, train/validation/test split and license or usage conditions.

3.5 Constraints

- Model performance depends strongly on training and test data quality.
- Small and distant objects may be difficult to detect.
- Low light and adverse weather can reduce image quality.
- Severe occlusion may cause missed detections or identity switches.
- High-resolution processing increases computational load.
- Large models can increase accuracy while increasing latency.
- Real-world road safety is more complex than visual detection alone.
- Cloud FPS measurements are not equivalent to embedded deployment performance.

3.6 Assumptions

- The camera has a reasonably stable forward-facing viewpoint.
- The video contains sufficient visual information for the target classes.
- Ground-truth labels are available for quantitative detection evaluation where possible.
- Temporary occlusions are short enough for track-management logic to be useful.
- The selected model and tracker can run in the chosen environment.
- Safety improvement is evaluated through measurable proxies rather than claiming actual accident reduction.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE / CONCEPTS

4.1 Image Acquisition

A camera converts the physical road scene into digital image frames. Frame rate, resolution, exposure and field of view influence downstream computer vision performance. A forward-facing camera is appropriate for road-scene monitoring because it captures objects and markings ahead of the vehicle.

4.2 Camera Calibration

Camera calibration estimates intrinsic parameters such as focal lengths and the optical center and can estimate distortion coefficients. A checkerboard calibration procedure is a common approach. The calibration parameters can be used with OpenCV to undistort frames before further geometric analysis.

4.3 Image Preprocessing

Preprocessing may include resizing, color conversion, denoising and contrast normalization. The objective is to make frames suitable for model inference without creating unnecessary computational overhead. Preprocessing should be evaluated because aggressive enhancement can sometimes reduce rather than improve model performance.

4.4 Object Detection

YOLO is selected because it is designed for efficient object detection. It predicts bounding boxes and class probabilities. The confidence threshold controls which detections are accepted. Non-maximum suppression or equivalent post-processing prevents multiple highly overlapping predictions from being reported as separate instances when they represent the same object.

4.5 Deep Learning

PyTorch provides the computational framework for neural-network inference and optional training/fine-tuning. GPU acceleration can reduce inference time. Batch size, image size, model architecture and hardware influence latency.

4.6 Traffic-Sign Recognition

Traffic-sign recognition provides semantic information beyond generic object detection. A sign detector can identify a sign location, while a classifier or fine-tuned detector can determine the sign category. Small signs, motion blur and occlusion are important challenges.

4.7 Road-Marking Recognition

Road markings provide information about lane structure and traffic rules. Lane boundaries can be detected using classical edge/region methods or learned segmentation/detection. Perspective transformation may help normalize the road surface for lane analysis.

4.8 Multi-Object Tracking

Tracking connects detections across time. ByteTrack associates high- and low-confidence detections using a tracking-by-detection approach. Track IDs enable the system to distinguish a persistent vehicle from a new vehicle entering the scene.

4.9 Occlusion Handling

Occlusion occurs when another object blocks part of the target. A track-management system can retain an object for a limited number of missing frames. When the object reappears, association can reconnect it to the previous identity. Long-term complete occlusion remains difficult.

4.10 Performance Metrics

Precision, recall and F1-score evaluate detection quality. Tracking can be evaluated using ID switches, track continuity, IDF1, MOTA and related metrics when ground truth is available. Latency measures responsiveness, while FPS measures processing throughput. A balanced evaluation is essential because a model with high accuracy but very high latency may not be practical for real-time use.

5. DESIGN / PROPOSED SOLUTION / METHODOLOGY

5.1A PROFESSIONAL SYSTEM ARCHITECTURE DIAGRAM

CAMERA / ROAD VIDEO

INPUT LAYER

Forward-facing camera
Road-scene video / image sequence



CALIBRATION & PREPROCESSING

Camera calibration → Lens distortion correction
Resize → Color conversion → Lightweight enhancement



PERCEPTION LAYER

YOLO: Vehicle & pedestrian detection
Traffic-sign recognition

Road-marking / lane recognition



TEMPORAL UNDERSTANDING

ByteTrack multi-object tracking
Track IDs + trajectory history
Occlusion handling / short-term track retention



SAFETY ANALYSIS

ROI monitoring
Pedestrian-entry / restricted-zone events
Safety-relevant event indicators



OUTPUT & EVALUATION

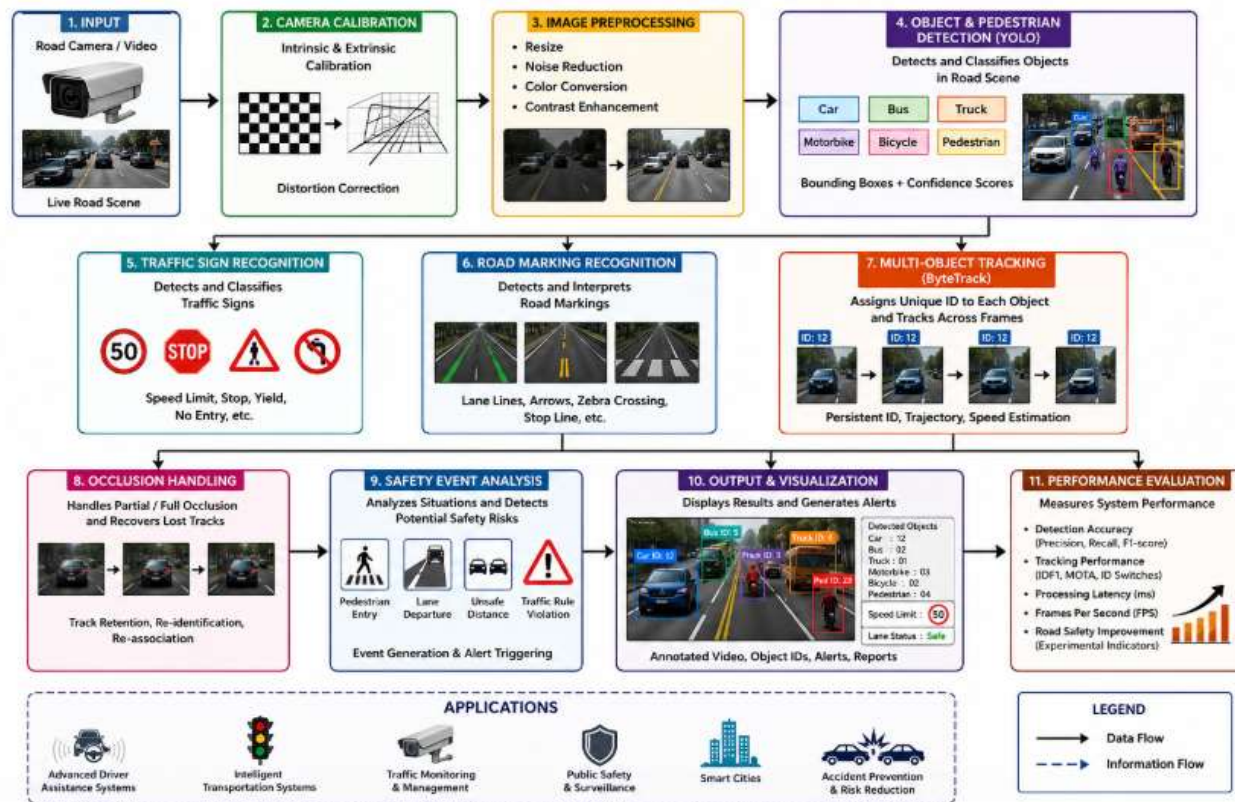
Annotated video + labels + confidence + IDs
Precision / Recall / F1
ID switches / continuity / IDF1 / MOTA
Latency / FPS + condition-wise comparison

INTELLIGENT TRANSPORTATION & PUBLIC SAFETY SUPPORT

Architecture note: the system is a perception and safety-monitoring prototype. It does not directly control steering, braking, or acceleration.

5.1 Overall System Architecture

The proposed system follows a sequential modular pipeline: video acquisition → camera calibration → preprocessing → YOLO object detection → sign recognition → road-marking recognition → ByteTrack multi-object tracking → occlusion handling → safety-event analysis → visualization → performance evaluation.



5.2 Module 1: Video Acquisition

The video module reads frames using OpenCV. The system records frame dimensions and source FPS where available. A frame counter is maintained so that detection and tracking results can be synchronized with the original video.

5.3 Module 2: Camera Calibration

Calibration images are captured using a known checkerboard pattern when the physical camera is available. OpenCV corner-detection functions are used to identify pattern points. Multiple views improve parameter estimation. The camera matrix and distortion coefficients are saved for later inference.

5.4 Module 3: Preprocessing

The frame is undistorted, resized to the model input size and converted into the expected image representation. Optional preprocessing is applied only when it improves measured performance. The pipeline records processing time so preprocessing overhead can be analyzed.

5.5 Module 4: Object and Pedestrian Detection

YOLO identifies target road users. Each detection contains a bounding box, confidence score and class. A confidence threshold is selected using validation experiments. The system prioritizes pedestrians because they are vulnerable road users and may require safety-specific monitoring.

5.6 Module 5: Road-Sign Recognition

A dedicated sign detector/classifier or a custom YOLO model is used depending on the dataset. The module identifies selected sign categories and passes them to the visualization layer. Sign confidence values should be logged for quantitative evaluation.

5.7 Module 6: Road-Marking Recognition

Road markings can be extracted using edge detection, region-of-interest processing and line fitting for a baseline, or a trained model for a more advanced implementation. The system may display lane boundaries and special markings such as stop lines or pedestrian crossings.

5.8 Module 7: ByteTrack Tracking

The detections are supplied to ByteTrack. Each active object receives a track ID. Track history can be stored to visualize trajectories. Tracking allows the system to distinguish persistent road users and can support motion-related safety analysis.

5.9 Module 8: Occlusion Handling

A maximum track age is used to avoid immediately deleting an object after one missed detection. If a target is missing for a short period, its previous state can be retained. Reappearance is matched to the existing track when association criteria are satisfied. The lost-frame threshold should be tuned using validation clips.

5.10 Module 9: Safety Event Analysis

A defined region of interest can represent a monitored lane or pedestrian crossing. The system can flag events such as a pedestrian entering a monitored area, a vehicle entering a restricted region or an object remaining in a danger zone. These are research-level indicators and should not be described as certified collision predictions.

5.11 Module 10: Visualization and Logging

The output displays bounding boxes, class labels, confidence scores, track IDs and optional warnings. A logging module stores frame number, processing time, FPS and safety-event counts. These logs are later used for graphs and validation.

6. ALGORITHM / PSEUDOCODE / FLOWCHART

6.1 Integrated Algorithm

9. Initialize video source and system configuration.
10. Load camera calibration parameters.
11. Load YOLO detector and sign/markings models.
12. Initialize ByteTrack.
13. Read a video frame.
14. Undistort and preprocess the frame.
15. Run YOLO detection.
16. Run traffic-sign recognition.
17. Run road-marking/lane detection.
18. Pass object detections to ByteTrack.
19. Update track identities and histories.
20. Retain short-lived missing tracks for occlusion handling.
21. Evaluate defined safety-event rules.
22. Draw annotations and FPS.
23. Store frame metrics and optional output video.
24. Repeat until the final frame.
25. Calculate detection, tracking and latency metrics.
26. Generate graphs and final observations.

6.2 Pseudocode

BEGIN

 Initialize video source

 Load calibration parameters

 Load YOLO model

 Load sign and marking models

 Initialize ByteTrack

WHILE video is available:

 frame $\leftarrow$ read frame

 start_time $\leftarrow$ current time

frame $\leftarrow$ undistort(frame)

frame $\leftarrow$ preprocess(frame)

detections $\leftarrow$ YOLO(frame)

signs $\leftarrow$ SignModel(frame)

markings $\leftarrow$ MarkingModel(frame)

tracks $\leftarrow$ ByteTrack.update(detections)

FOR each active track:

 update track history

 IF detection temporarily missing:

 retain track up to max_age

safety_events $\leftarrow$ evaluate_rules(tracks, signs, markings)

draw boxes, labels, IDs, signs and markings

calculate latency and FPS

save/display frame

END WHILE

calculate precision, recall and F1

calculate tracking metrics

calculate latency/FPS

generate graphs

END

6.3A PROFESSIONAL INTEGRATED FLOW CHART

1. START



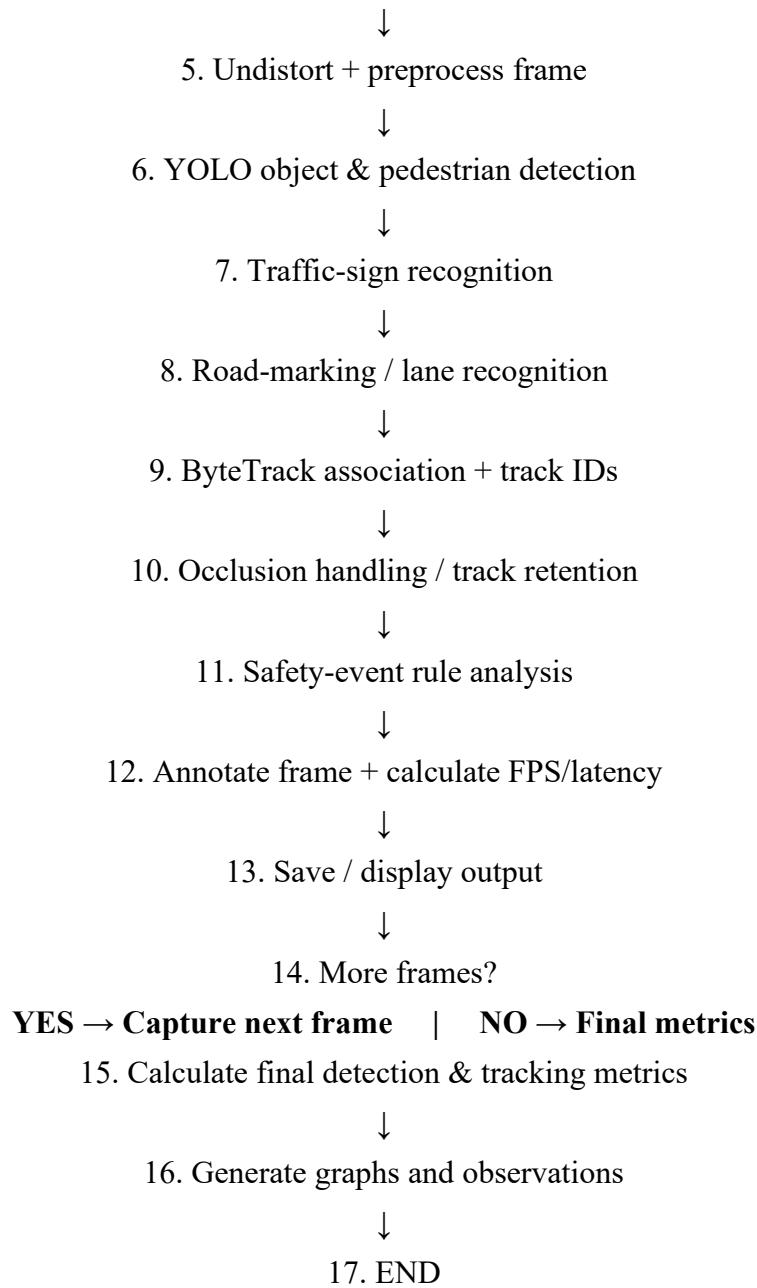
2. Load road video / camera stream



3. Load camera calibration parameters



4. Capture next frame



Flowchart note: the decision branch prevents the pipeline from terminating until the complete test sequence has been processed.

6.3 Flowchart

START → Load Video → Load Calibration → Preprocess Frame → YOLO Object Detection → Sign Recognition → Road-Marking Detection → ByteTrack → Occlusion Handling → Safety Event Analysis → Annotate Output → Calculate Latency/FPS → Save Result → More Frames? → YES: Repeat / NO: Calculate Final Metrics → Generate Graphs → END

6.4 Decision Logic for Occlusion

If a detection is present, update the track normally. If a previously active object is not detected in the current frame, increase its lost-frame counter. If the lost-frame counter remains below the selected maximum, keep the track temporarily. If the object reappears and association succeeds, reset the counter. If the counter exceeds the maximum, terminate the track. This logic limits unnecessary ID fragmentation while preventing indefinitely stale tracks.

7. IMPLEMENTATION / SOURCE CODE AND ENVIRONMENT / TOOLS USED

7.1 Google Colab Environment

Google Colab is recommended because it provides a convenient notebook environment and can provide GPU acceleration. The notebook should begin by checking Python and GPU availability and then installing or importing the required packages.

7.3 Representative OpenCV Video Loop

```
import cv2
from time import perf_counter

cap = cv2.VideoCapture("road_video.mp4")

while cap.isOpened():
    ok, frame = cap.read()
    if not ok:
        break

    start = perf_counter()

    # Camera correction:
    # frame = cv2.undistort(frame, camera_matrix, dist_coeffs)

    # Detection:
    # detections = yolo_model(frame)
```

```

# Tracking:

# tracks = tracker.update(detections)


elapsed = perf_counter() - start
fps = 1.0 / elapsed if elapsed > 0 else 0.0


cv2.putText(frame, f'FPS: {fps:.2f}', (20, 35),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)


cv2.imshow("Road Safety System", frame)


if cv2.waitKey(1) & 0xFF == ord("q"):
    break


cap.release()
cv2.destroyAllWindows()

```

7.4 Implementation Notes

The exact YOLO and ByteTrack APIs vary by implementation. Therefore, the final GitHub code should document the exact package/model versions used. The report should not claim that a module was implemented if it was only described conceptually. Actual screenshots and measurements must correspond to the submitted source code.

7.5 Tools Used

Tool	Use in Project
Python	Core implementation
OpenCV	Calibration, preprocessing, video and drawing
YOLO	Object/pedestrian detection
PyTorch	Model execution
ByteTrack	Multi-object tracking
NumPy	Numerical processing

Pandas	Results tables
Matplotlib	Graphs
Google Colab	Experiment environment
GitHub	Version control and final submission

8. TEST CASES AND EXPECTED / ACTUAL RESULTS

Testing should be performed using representative clips. Actual values must be filled after execution; placeholders below prevent unsupported claims.

ID	Scenario	Expected Result	Actual Result	Status
TC01	Daytime + light traffic	High detection stability and continuous tracks	To be measured	Pending
TC02	Daytime + heavy traffic	Multiple objects detected with manageable ID switches	To be measured	Pending
TC03	Pedestrian crossing	Pedestrian detected and tracked	To be measured	Pending
TC04	Vehicle partially occluded	Track retained where possible	To be measured	Pending
TC05	Traffic sign visible	Correct sign category recognized	To be measured	Pending
TC06	Clear lane markings	Lane/markings detected	To be measured	Pending
TC07	Low-light scene	System remains functional with measurable degradation	To be measured	Pending

TC08	Rain/haze	Performance degradation quantified	To be measured	Pending
TC09	Crowded scene	Tracking maintains most identities	To be measured	Pending
TC10	Long-duration video	No major processing failure	To be measured	Pending

8.1 Functional Testing

Functional testing verifies that each module operates correctly before integrated evaluation. Calibration output, detector output, sign output, marking output and tracking output should be inspected independently. This reduces the chance of attributing an integration problem to the wrong module.

8.2 Performance Testing

Performance testing measures accuracy and speed. Each condition should use the same metric definitions. If multiple model configurations are compared, the same test clips and evaluation procedure should be used to make the comparison fair.

9.1 Screenshot Documentation Format

For every important screenshot, include a figure number and a one-sentence explanation. The screenshot should show enough of the interface to establish that the output was actually generated by the submitted implementation. Avoid using unrelated stock images as execution evidence.

9.2 Output Interpretation



A successful integrated output should show road objects with bounding boxes, labels and confidence values. Tracked objects should have persistent IDs. Sign and marking results should be visually distinct. The final screen may also show FPS and safety-event indicators. If an object is missed or an ID switches during occlusion, the example can be included as evidence for the limitations section.

10.0A Evaluation Standard and Result Interpretation

DETECTION QUALITY

Use Precision, Recall and F1-score from labeled test data.

Higher Precision → fewer false positives.

Higher Recall → fewer missed road users.

TRACKING QUALITY

Use ID switches, track continuity, IDF1 and MOTA where ground truth permits.

Fewer ID switches and higher continuity indicate more stable temporal tracking.

REAL-TIME PERFORMANCE

Measure average/median frame latency and FPS separately for CPU and GPU.

Do not mix model warm-up time with steady-state processing.

ROAD-SAFETY PROXY

Compare timely detection of defined safety events against a detection-only baseline.

Do not claim actual accident reduction from an academic prototype.

10. RESULTS AND VALIDATION

10.1 Detection Metrics

For detection evaluation, true positives, false positives and false negatives should be counted using the chosen matching criterion. Precision indicates how many reported detections are correct. Recall indicates how many actual objects were detected. F1-score provides a single balanced indicator.

Metric	Formula / Definition	Interpretation	Actual Value
Precision	$TP/(TP+FP)$	Detection correctness	TBD
Recall	$TP/(TP+FN)$	Detection completeness	TBD
F1-score	$2PR/(P+R)$	Balance of precision and recall	TBD
Latency	Total processing time / number of frames	Average frame processing time	TBD
FPS	Frames processed / elapsed time	Processing throughput	TBD

10.2 Tracking Metrics

When tracking ground truth is available, use standard metrics such as IDF1, MOTA, ID switches and track fragmentation. If complete ground truth is not available, report track continuity and manually verified ID-switch examples, clearly identifying the evaluation limitation.

Metric	Meaning	Desired Direction
--------	---------	-------------------

ID Switches	Number of times an object receives an incorrect new identity	Lower
Track Continuity	Percentage/time for which a target remains correctly tracked	Higher
IDF1	Identity-based tracking accuracy	Higher
MOTA	Combined multi-object tracking accuracy measure	Higher

10.3 Condition-Based Evaluation

Condition	Traffic	Precision	Recall	F1	Tracking	Latency
Clear daytime	Light	TBD	TBD	TBD	TBD	TBD
Clear daytime	Heavy	TBD	TBD	TBD	TBD	TBD
Low light	Light	TBD	TBD	TBD	TBD	TBD
Low light	Heavy	TBD	TBD	TBD	TBD	TBD
Rain/Haze	Light	TBD	TBD	TBD	TBD	TBD
Rain/Haze	Heavy	TBD	TBD	TBD	TBD	TBD

10.4 Latency Analysis

Latency should be measured over a long enough sequence to avoid drawing conclusions from a single frame. Report average and median latency where possible. CPU and GPU results should be labeled separately. Model warm-up time should not be mixed with steady-state frame-processing time.

10.5 Safety Improvement Evaluation

Because this is an academic prototype, it is not scientifically valid to claim a percentage reduction in actual road accidents without a large real-world study. A suitable experimental proxy is timely detection of safety-relevant events. For example, compare the percentage of pedestrian-entry events detected before a predefined time threshold with a detection-only baseline. The threshold and baseline must be fixed before testing.

10.6 Suggested Graphs

- Graph 1: Precision, recall and F1-score by traffic condition.
- Graph 2: Tracking ID switches by traffic condition.
- Graph 3: Average latency by environmental condition.
- Graph 4: FPS comparison for CPU and GPU.
- Graph 5: Detection performance versus traffic density.
- Graph 6: Safety-event detection rate for baseline versus integrated system.

11. ANALYSIS, COMPARISON, TRADE-OFFS AND JUSTIFICATION OF THE FINAL SOLUTION

11.1 Accuracy versus Latency

Increasing model complexity can improve detection quality but generally increases inference cost. A smaller model may be more suitable for near-real-time processing but can miss small or distant objects. Therefore, model selection should be based on a joint view of accuracy and latency.

11.2 Detection Threshold Trade-off

A high confidence threshold can reduce false positives but may increase false negatives. A lower threshold can improve recall but can introduce more incorrect detections. The threshold should be selected using validation data rather than chosen only for visually attractive output.

11.3 Tracking Trade-off

Keeping tracks alive for more missing frames can improve continuity during temporary occlusion. However, if the object has left the scene, excessive track retention may create false persistence. The lost-frame threshold therefore requires tuning.

11.4 Comparison of Approaches

Approach	Strength	Weakness	Use in Project
Detection only	Simple and fast	No temporal identity	Baseline
Detection + ByteTrack	Persistent IDs and motion continuity	Depends on detector quality	Recommended
Camera + vision	Lower sensor cost, rich appearance	Weather/lighting sensitivity	Primary prototype
Camera + LiDAR	Depth + appearance	Higher cost and complexity	Future extension
Camera + Radar	Motion/range complement	Sensor fusion required	Future extension

11.5 Why the Proposed Combination is Suitable

Python provides rapid development, OpenCV provides mature image/video functionality, YOLO offers efficient object detection, PyTorch supports deep-learning execution, and ByteTrack provides practical multi-object tracking. Google Colab reduces setup effort and enables GPU-based experimentation.

Together, these tools are appropriate for an academic prototype that emphasizes integration and evaluation.

11.6 Practical Relevance

The system can support research into driver-assistance, traffic surveillance, pedestrian monitoring, road-infrastructure analytics and intelligent transportation. It can also provide a foundation for future sensor-fusion systems. However, production use requires validation beyond the scope of this assignment.

12. BROADER CONSIDERATIONS / SDG RELEVANCE

12.1 Intelligent Transportation Systems

Intelligent transportation systems use sensing, communication and computation to improve transportation efficiency and safety. The proposed system contributes a perception layer by converting visual road information into structured data. Such data can be used by higher-level traffic-monitoring or driver-assistance components.

12.2 Public Safety

The ability to identify pedestrians, vehicles and road signs can support safety monitoring. Automated detection can operate continuously and can help identify events that deserve attention. Human oversight remains important for high-risk decisions.

12.3 SDG 11 – Sustainable Cities and Communities

The project is relevant to SDG 11 because safer transportation and improved urban infrastructure contribute to safer and more sustainable communities. Computer vision can be one component of smart-city transportation systems.

12.4 SDG 9 – Industry, Innovation and Infrastructure

The project also relates to SDG 9 through the application of artificial intelligence and computer vision to transportation infrastructure and technological innovation.

12.5 Privacy

Road cameras may capture faces, vehicle identifiers and other potentially sensitive information. A practical deployment should minimize data collection, restrict access, apply appropriate anonymization where possible and comply with applicable privacy requirements.

12.6 Bias and Fairness

A model trained primarily on one geographic region, road design or weather condition may perform differently elsewhere. Evaluation should therefore include diverse scenes, object sizes, lighting conditions and traffic patterns.

12.7 Safety-Critical Deployment

This prototype should be considered a perception research system, not a certified safety-critical controller. A production system would require redundancy, fail-safe behavior, formal validation, extensive road testing and regulatory approval.

13. CONCLUSION, LIMITATIONS AND POSSIBLE IMPROVEMENTS

13.1 Conclusion

The proposed computer vision–based autonomous road safety system integrates multiple perception functions into a unified pipeline. Camera calibration improves the geometric quality of visual input, YOLO provides object and pedestrian detection, road-sign and road-marking recognition adds semantic road information, and ByteTrack provides temporal identity. Occlusion handling improves continuity when objects are briefly hidden.

The evaluation framework emphasizes detection accuracy, tracking performance and processing latency while also considering safety-relevant indicators. Testing across traffic and environmental conditions provides a more realistic assessment of robustness than a single clear-weather experiment. The project demonstrates the value of computer vision as a supporting technology for intelligent transportation and public safety.

13.2 Limitations

- Performance depends on the selected dataset and model.
- Small and distant objects may be missed.
- Severe weather and low illumination can degrade detection.
- Long-term occlusion can cause identity loss.
- Road markings may be faded or physically damaged.
- Camera-only systems lack direct depth/range sensing.
- Cloud GPU performance is not equivalent to embedded deployment.
- Actual accident reduction cannot be inferred from a small academic experiment.
- Privacy and cybersecurity issues must be addressed in real deployments.

13.3 Future Improvements

- Fine-tune YOLO on local road and traffic-sign datasets.
- Use semantic/instance segmentation for road markings.
- Add depth estimation or stereo cameras.
- Fuse camera with radar or LiDAR.
- Use stronger re-identification for long-term tracking.
- Optimize the model through quantization or suitable inference acceleration.
- Deploy on an edge AI platform.
- Add calibrated risk estimation and time-to-collision research.
- Expand evaluation to more cities, weather conditions and traffic patterns.
- Introduce continuous monitoring for model drift after deployment.

14. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

The contribution table must be completed according to actual work. The final report should agree with GitHub commits, notebooks, source files and screenshots.

Member	Primary Role	Activities	Evidence
Member 1	Data & Calibration	Dataset preparation, camera calibration, preprocessing	Code/screenshots/commits
Member 2	Detection & Recognition	YOLO detection, pedestrian and sign recognition	Code/screenshots/commits
Member 3	Tracking & Evaluation	ByteTrack, occlusion handling, metrics and graphs	Code/screenshots/commits

14.1 Suggested Work Distribution

- Member 1: Dataset preparation, calibration and preprocessing.
- Member 2: Object/pedestrian detection and road-sign recognition.
- Member 3: Road marking, ByteTrack tracking and occlusion handling.
- All members: Integration, testing, results, report and presentation.

14.2 Evidence of Contribution

Each member should maintain evidence such as GitHub commits, notebook sections, source-code modules, test results and screenshots. This makes the contribution section verifiable and avoids ambiguity during evaluation.

15. REFERENCES

Use the exact versions and sources used in the final implementation. The following are suitable foundational references for the project.

27. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
28. Jocher, G. et al. Ultralytics YOLO documentation and model resources.
29. Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems.
30. Zhang, Y. et al. (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. European Conference on Computer Vision.
31. Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. European Conference on Computer Vision.
32. Geiger, A., Lenz, P., and Urtasun, R. (2012). Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite. CVPR.
33. OpenCV documentation used for camera calibration, image processing and video operations.
34. PyTorch documentation used for model execution and GPU support.
35. Documentation for the exact traffic-sign dataset used in the implementation.
36. Documentation for the exact road/lane-marking dataset used in the implementation.

15.1 Software/Tool Record

Software	Version Used	Purpose
Python	To be filled	Programming
OpenCV	To be filled	Computer vision
PyTorch	To be filled	Deep learning
YOLO package/model	To be filled	Detection
ByteTrack implementation	To be filled	Tracking
Google Colab	To be filled	Execution
CUDA/GPU	To be filled if applicable	Acceleration

15.2 Dataset Record

Dataset	Classes / Content	Split	Source / License
Dataset 1	To be filled	Train/Val/Test	To be filled
Dataset 2	To be filled	Train/Val/Test	To be filled

16. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS – DETAILED RECORD

We confirm that the individual contribution details entered in Section 14 represent the actual work completed by each group member. Implementation results, screenshots, numerical metrics and observations included in the final submission should be traceable to the executed project files and recorded experiments.

Member Signature: M Jeevitha Date: 2-9-26

